

SPIN - Seminar

An Improvement in Formal Verification – Static Reduction

Formal Methods for AI-Based Systems Engineering (FMAISE)

Robert Li & William Tossici (1983696, 1844504)

Academic Year 2025/2026



SAPIENZA
UNIVERSITÀ DI ROMA



Table of Contents

1 Introduction

► Introduction

► Definitions

► Reduction Algorithm

► Implementation

► Performance



Introduction: The Verification Challenge

1 Introduction

- **The Problem:** Standard depth-first search explores redundant execution sequences due to interleaved independent actions.
- **Dynamic Reduction:** Computes redundancies at run-time, but CPU and memory overhead can often outweigh the benefits.
- **Static Reduction (Proposed):** Performs redundancy computations *before* the search begins.
- **Key Advantages:** Eliminates run-time overhead while preserving both safety and liveness properties for LTL model checking.



Table of Contents

2 Definitions

► Introduction

► **Definitions**

► Reduction Algorithm

► Implementation

► Performance



System Modeling: Labeled Transition Systems (LTS)

2 Definitions

- **LTS Definition:** A triple $\{S, s_0, T\}$ modeling concurrent system behavior.
- S : Finite set of states.
- s_0 : Initial state.
- T : Finite set of transitions representing atomic process statements.
- **Key Functions:**
 - $Cond(a)$: Subset of states where statement a is enabled.
 - $Act(a, s)$: State reached after executing a from state s .



Statement Independence

2 Definitions

- **Independence at state s :** Two statements a and b are independent if:
 1. Both are enabled in s .
 2. Execution of one does not disable the other.
 3. Execution order does not change the final state (commutative).
- **Global Independence:** Statements are independent in *every* state where simultaneously enabled.
- **Constraint:** Two statements from the *same* sequential process can **never** be independent.



Formal Verification via Synchronous Product

2 Definitions

- **Properties:** Expressed using next-time-free Linear Temporal Logic (LTL).
- **Büchi Automaton (G):** The LTL formula is formalized as an automaton with predicate labels and acceptance states.
- **Synchronous Product ($F \times G$):** The combined LTS of the concurrent system (F) and the Automaton (G).
- **Goal:** Detect "acceptance cycles" in $F \times G$ to prove or disprove property satisfaction.



Safe and Observable Statements

2 Definitions

- **Observable Statement:** Execution alters the truth-value of a proposition monitored by the Büchi Automaton.
- **Safe Statement:**
 - Non-observable to the Automaton.
 - Globally independent from all statements in other processes.
- **Conditionally Safe Statement:** Safe only in states where a specific predefined condition $P(s)$ holds.



Table of Contents

3 Reduction Algorithm

► Introduction

► Definitions

► Reduction Algorithm

► Implementation

► Performance



Initialization

3 Reduction Algorithm

```
1 start_search( $s_0$ )
2 { derive and optimize transition structures
3   enter  $s_0$  into Statespace;
4   push  $s_0$  onto Stack;
5   Dfs(1);
6 }
```



Expansion Step for the Sequential Processes

3 Reduction Algorithm

```
1  dfs(N)
2  {    s = top(Stack);                                //current state
3      for each sequential process i                    //
4      {    nxt = all transitions in F enabled in s with Pid(t) = i
5          for all t in nxt
6          {    s' = successor of s after t;
7              if {s',N} NOT in Statespace                //new state
8              {    enter {s',N} into Statespace;
9                  push s' onto Stack;
10                 Dfs(N);                                //Buchi Automaton
11          }    }    }
12      pop s from Stack
13 }
```



Interleaved Transitions of Büchi Automaton

3 Reduction Algorithm

```
1  Dfs(N)
2  {    s = top(Stack);                                //current state
3      nxt = all transitions in G enabled in s;        //Buchi Automaton
4      for all t in nxt
5      {    s' = successor of s after t;
6          if {s',N} NOT in Statespace
7          {    enter {s',N} into Statespace;
8              push s' onto Stack;
9              dfs(N);                                //synchronous product F×G
10         }    }
11     pop s from Stack
12 }
```



Extension for Cycle Detection

3 Reduction Algorithm

```
1  Dfs(N)
2  {    s = top(Stack);                                //current state
3      nxt = all transitions in G enabled in s;        //Buchi Automaton
4      for all t in nxt
5      {    s' = successor of s after t;
6          if N == 2 and s' == seed                    //cycle detection
7          {    report acceptance cycle                //error report
8              return
9          }
10         if {s',N} NOT in Statespace
11         {    enter {s',N} into Statespace;
12             push s' onto Stack;
13             dfs(N);
14             if N == 1 and s is an accepting state in G
15             {    seed = s
16                 dfs(2)                                //second dfs
17             }
18         }    }
19     pop s from Stack
20 }
```



Static Reduction: Core Concepts

3 Reduction Algorithm

- **Objective:** Find the smallest set of transitions to explore while preserving both safety and liveness properties.
- **The Prescan Phase:** The algorithm pre-scans processes to identify one that can execute only *safe* or *conditionally safe* transitions.
- **Dependency Rule:** If a transition is selected for exploration, all simultaneously enabled transitions that are *not* independent from it must also be selected.
- **The Reduction Proviso:** A crucial safeguard. The reduction attempt is only valid if none of the explored transitions lead back to a state currently in the search stack.
- **Fallback:** If the proviso fails, the algorithm explores transitions from another process, up to a full exploration of all enabled transitions if necessary.



Reduced Expansion Step

3 Reduction Algorithm

```
1  dfs(N)
2  {    s = top(Stack);
3      order processes;                                //using safety as ordering principle
4      for each sequential process i
5      {    boolean NotInStack = true
6          boolean AtLeastOneSuccessor = false
7          nxt = all transitions t in F enabled in s with Pid(t) = i
8          for all t in nxt
9          {    s' = successor of s after t;
10             if {s',N} NOT in Statespace
11             {    enter {s',N} into Statespace;
12                 push s' onto Stack;
13                 Dfs(N);
14             }    else if s' in Stack                                //reduction proviso
15                 NotInStack = false
16                 AtLeastOneSuccessor = true
17             }
18             if AtLeastOneSuccessor  $\wedge$  NotInStack                                //safe
19                 break                                                //from the loop over processes
20         }
21     pop s from Stack
22 }
```



Table of Contents

4 Implementation

► Introduction

► Definitions

► Reduction Algorithm

► **Implementation**

► Performance



PROMELA Implementation: Safe Operations

4 Implementation

- **Unconditionally Safe Statements:**
 1. Access to exclusively local variables.
 2. Receive from a queue with *exclusive receive-access*.
 3. Send to a queue with *exclusive send-access*.
 4. `nfull(q)` test (requires exclusive send-access).
 5. `nempty(q)` test (requires exclusive receive-access).
- **Conditionally Safe:** The above operations become conditionally safe when used as guards in selection structures.
- **Guard Conditions:** They require specific logical conditions to be true (e.g., `nempty(q)` for receives, `nfull(q)` for sends) to remain safe.



PROMELA Implementation: Extensions & Assertions

4 Implementation

- **Language Extensions:** Introduced `nfull()` and `nempty()` primitives. Parser rules prevent negations to keep static analysis reliable.
- **Scope Rules:** The *never claim* (Büchi Automaton) can only access global objects. Any non-exclusive reference to a queue automatically marks operations as observable.
- **Handling Dynamic Processes:**
 - Queue exclusivity is declared via explicit assertions (`xr` for receive, `xs` for send).
 - **Runtime Validation:** Assertions are checked at runtime. If falsely claimed by the programmer, the reduced search will eventually execute the conflicting statement and report an assertion violation.



Table of Contents

5 Performance

- ▶ Introduction
- ▶ Definitions
- ▶ Reduction Algorithm
- ▶ Implementation
- ▶ Performance



Static vs. Dynamic

5 Performance

Protocol	Algorithm	States	Transitions	Time(sec.)	Memory (Mb)
Best-Case	Non-Reduced	100,001	450,002	13.2	4.3
	Static Reduction	47	47	(<0.1)	1.0
	Dynamic Reduction	47	47	0.1	1.4
Worst-Case	Non-Reduced	100,001	450,002	14.5	5.0
	Static Reduction	100,001	450,002	16.7	5.1
	Dynamic Reduction	100,001	450,002	84.5	5.3
Tpc	Non-Reduced	3,918,286	11,762,426	630.6	268.4
	Static Reduction	391,534	466,753	30.6	26.2
	Dynamic Reduction	267,204	295,395	131.4	18.9
Snoopy	Non-Reduced	91,920	305,460	14.4	11.5
	Static Reduction	16,279	23,532	1.7	3.2
	Dynamic Reduction	7,158	8,459	6.8	2.6
Pftp	Non-Reduced	417,321	1,244,865	73.2	62.3
	Static Reduction	53,244	67,901	6.8	9.3
	Dynamic Reduction	125,718	163,459	105.5	20.6
Leader	Non-Reduced	45,885	185,032	8.1	9.6
	Static Reduction	79	79	0.1	1.1
	Dynamic Reduction	79	79	0.2	1.4



Performance Evaluation: Key Findings

5 Performance

- **Properties Preserved:** Static reduction preserves both safety and liveness, whereas the tested dynamic method preserves only safety.
- **Overhead vs. State Space:** Dynamic methods may explore fewer states by exploiting runtime data, but static reduction benefits from near-zero computational overhead.
- **Run-Time Results:** Despite sometimes exploring more states, static reduction achieved the shortest run-time across all tested real-world protocols.
- **Worst-Case Behavior:** Static reduction adds a negligible time penalty compared to standard search, whereas dynamic reduction can suffer heavily from runtime overhead.



Exploiting Structure

5 Performance

- **A Priori Knowledge:** Static reduction rules are known before execution.
- **Model Rewriting:** Protocols can be intentionally restructured to favor the algorithm without altering functionality.
- **Case Study: IEEE 802.2 LLC Protocol**
 - *Original (Shared Queues):* Standard: 1,851,049 states → Reduced: 111,159 states.
 - *Revised (Exclusive Queues per Source):* Standard: 19,407 states → Reduced: 2,000 states.
- **Result:** Achieved 3 orders of magnitude improvement, enabling “Structured Design Validation”.



SPIN - Seminar *Thank you for listening!*

Any questions?



Why Next-Time-Free LTL?

5 Performance

- **The Challenge:** Static reduction actively skips or reorders “safe” (non-observable) transitions to reduce the search space.
- **The “Next-Time” ($\bigcirc$) Operator:** It rigidly evaluates the very next micro-step. If the reduction algorithm omits a safe step, formulas using this operator would break.
- **The Solution:** Next-time-free LTL focuses only on the correct order of *observable* events, ignoring irrelevant intermediate steps.
- **Core Property:** The set of sequences satisfying these formulas is mathematically “closed under stuttering”.

Example: Stuttering Equivalence

- *Original sequence:* State A $\rightarrow$ (safe step) $\rightarrow$ State A $\rightarrow$ (observable) $\rightarrow$ State B
- *Reduced sequence:* State A $\rightarrow$ (observable) $\rightarrow$ State B
- *With “Next-Time”:* “Next state is B” is FALSE in the original, but TRUE in the reduced.
- *Next-Time-Free:* “Eventually B” is TRUE in both cases.



The `never` Claim in PROMELA

5 Performance

- **What it is:** In PROMELA, the `never` claim represents the Büchi Automaton, acting as a monitor for correctness properties.
- **Scope Rules:** To ensure reliable analysis, the `never` claim is strictly restricted to accessing only *global* objects.
- **Invisibility (Safe Status):** Operations on purely local variables or exclusive queues cannot be seen by the `never` claim, preserving their “safe” status.
- **Observability:** Any non-exclusive reference to a queue makes it global. The `never` claim can then observe it, which instantly marks the operation as “observable” and not safe.